

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Code:

SingletonTest.java

```
package com.result.Singleton;
public class SingletonTest {

    public static void main(String[] args) {

        // Get Logger instance
        Logger logger1 = Logger.getInstance();

        // Use logger
        logger1.log("Application Started");

        // Get Logger instance again
        Logger logger2 = Logger.getInstance();

        logger2.log("User Logged In");

        // Check whether both references point to same object
        if (logger1 == logger2) {
            System.out.println("Only One Logger Instance Exists");
        } else {
            System.out.println("Multiple Instances Exist");
        }
    }
}
```

Logger.java

```
package com.result.Singleton;
class Logger {

    // Step 1: Create a private static instance
    private static Logger instance;

    // Step 2: Private constructor
    private Logger() {
        System.out.println("Logger Instance Created");
    }

    // Step 3: Public method to get the instance
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
    }
}
```

```

        return instance;
    }

    // Logging method
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}

```

Output:

The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure: SingletonPatternExample, JRE System Library (JavaSE-21), src, com.result.Singleton, Logger.java, SingletonTest.java, and module-info.java. The main editor window shows the SingletonTest.java file with the following code:

```

1 package com.result.Singleton;
2 public class SingletonTest {
3
4     public static void main(String[] args) {
5
6         // Get Logger Instance
7         Logger logger1 = Logger.getInstance();
8
9         // Use logger
10        logger1.log("Application Started");
11
12        // Get logger instance again
13        Logger logger2 = Logger.getInstance();
14
15        logger2.log("User Logged In");
16
17        // Check whether both references point to same object
18        if (logger1 == logger2) {
19            System.out.println("Only One Logger Instance Exists");
20        } else {
21            System.out.println("Multiple Instances Exist");
22        }
23    }
24 }

```

The Console window at the bottom shows the output of the program:

```

Logger Instance Created
LOG: Application Started
LOG: User Logged In
Only One Logger Instance Exists

```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Code:

Document.java

```
package com.result.factory;

public interface Document {
    void open();
}
```

DocumentFactory.java

```
package com.result.factory;

public abstract class DocumentFactory {

    public abstract Document createDocument();}
```

ExcelDocument.java

```
package com.result.factory;

public class ExcelDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Excel Document");
    }
}
```

ExcelFactory.java

```
package com.result.factory;

public class ExcelFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}
```

PdfDocument.java

```
package com.result.factory;  
  
public class PdfDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening PDF Document");  
    }  
}
```

PdfFactory.java

```
package com.result.factory;  
  
public class PdfFactory extends DocumentFactory {  
  
    @Override  
    public Document createDocument() {  
        return new PdfDocument();  
    }  
}
```

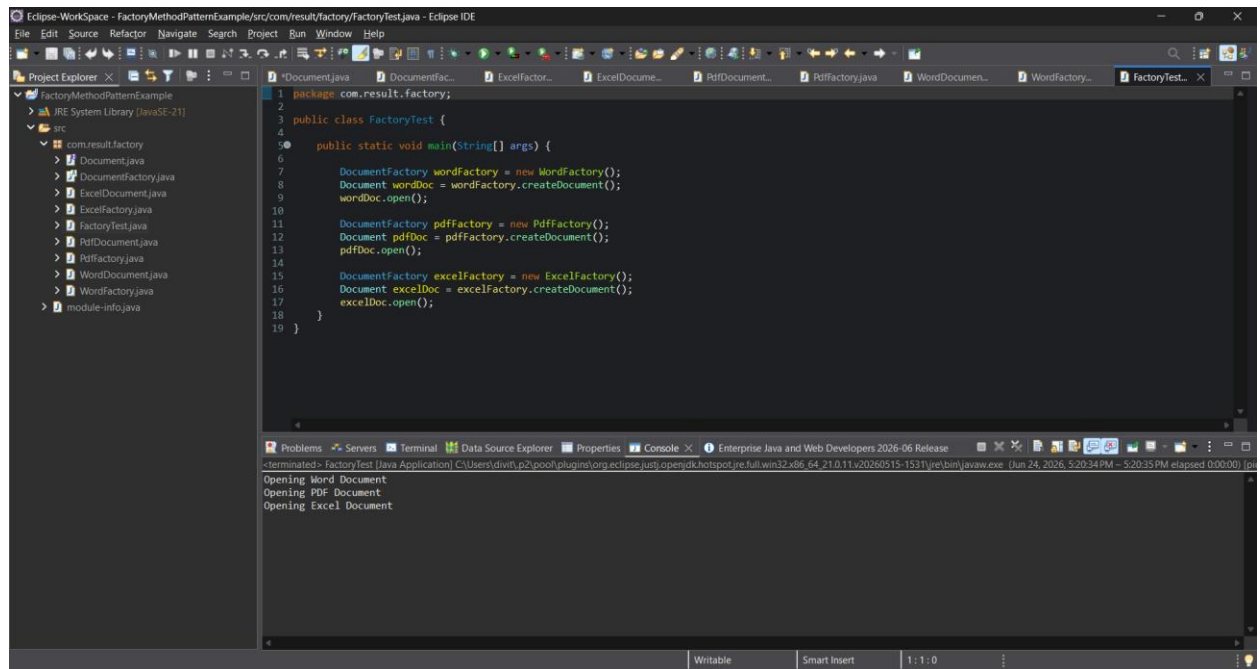
WordDocument.java

```
package com.result.factory;  
  
public class WordDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening Word Document");  
    }  
}
```

WordFactory.java

```
package com.result.factory;  
  
public class WordFactory extends DocumentFactory {  
  
    @Override  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}
```

Output:



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'FactoryMethodPatternExample', including the 'src' directory and the 'com.result.factory' package. The main editor window shows the 'FactoryTest.java' file with the following code:

```
1 package com.result.factory;
2
3 public class FactoryTest {
4
5     public static void main(String[] args) {
6
7         DocumentFactory wordFactory = new WordFactory();
8         Document wordDoc = wordFactory.createDocument();
9         wordDoc.open();
10
11         DocumentFactory pdfFactory = new PdfFactory();
12         Document pdfDoc = pdfFactory.createDocument();
13         pdfDoc.open();
14
15         DocumentFactory excelFactory = new ExcelFactory();
16         Document excelDoc = excelFactory.createDocument();
17         excelDoc.open();
18     }
19 }
```

The Console window at the bottom shows the output of the program:

```
terminated> FactoryTest [Java Application] C:\Users\dilip\p2\pool\plugins\org.eclipse.justi.openjdk hotspot.jre.full.win32.x86_64-21.0.11.v20200515-1531\jre\bin\javaw.exe [Jun 24, 2026, 5:20:34 PM - 5:20:35 PM elapsed 0:00:00] jpl
Opening Word Document
Opening PDF Document
Opening Excel Document
```

Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

Code:

Computer.java

```
package com.result.builder;

public class Computer {

    private String cpu;
    private int ram;
    private int storage;
    private String graphicsCard;

    // Private Constructor
    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
        this.graphicsCard = builder.graphicsCard;
    }

    public void display() {
        System.out.println("CPU : " + cpu);
        System.out.println("RAM : " + ram + " GB");
        System.out.println("Storage : " + storage + " GB");
        System.out.println("Graphics Card : " + graphicsCard);
        System.out.println();
    }

    // Static Nested Builder Class
    public static class Builder {

        private String cpu;
        private int ram;
        private int storage;
        private String graphicsCard;

        public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        public Builder setRAM(int ram) {
            this.ram = ram;
            return this;
        }

        public Builder setStorage(int storage) {
```

```

        this.storage = storage;
        return this;
    }

    public Builder setGraphicsCard(String graphicsCard) {
        this.graphicsCard = graphicsCard;
        return this;
    }

    public Computer build() {
        return new Computer(this);
    }
}

```

BuilderTest.java

```

package com.result.builder;

public class BuilderTest {

    public static void main(String[] args) {

        Computer gamingPC = new Computer.Builder()
            .setCPU("Intel i9")
            .setRAM(32)
            .setStorage(1000)
            .setGraphicsCard("NVIDIA RTX 4080")
            .build();

        Computer officePC = new Computer.Builder()
            .setCPU("Intel i5")
            .setRAM(8)
            .setStorage(256)
            .build();

        System.out.println("Gaming PC Configuration");
        gamingPC.display();

        System.out.println("Office PC Configuration");
        officePC.display();
    }
}

```